C++ Codebook

This codebook contains commonly used algorithms and utilities optimized for competitive programming.

1. Basic Utilities

IO Speedup (cin, cout)

```cpp
cin.sync_with_stdio(0);
cin.tie(0);
```

Buffered Output (Defer Flushing Until Program End)

```cpp
ios::sync_with_stdio(false);
cin.tie(NULL);
```

Test With File

```
g++ main.cpp -o main
main < input.in > output.out
```

Write File

```cpp
#include <iostream>
#include <fstream>
#include <string>

int main() {
    std::ofstream outFile("example.txt");
    if (!outFile.is_open()) {
        std::cerr << "Error opening file for
        ↪ writing!" << std::endl;
        return 1;
    }
    outFile << "Hello, World!\n";
    outFile << "This is a line in a text file.\n";
    outFile << 123 << " " << 45.67 << "\n";
    outFile.close();
    return 0;
}
```

Read File

```cpp
#include <iostream>
#include <fstream>
#include <string>

int main() {
    std::ifstream inFile("example.txt");
    if (!inFile.is_open()) {
        std::cerr << "Error opening file for
        ↪ reading!" << std::endl;
        return 1;
    }
    std::string line;
    while (std::getline(inFile, line)) {
        std::cout << line << std::endl;
    }
    inFile.close();
    return 0;
}
```

Greatest Common Divisor (GCD)

```cpp
int GCD(int x, int y) {
    if (!x) return y;
    if (!y) return x;
    while ((x %= y) && (y %= x));
    return x + y;
}
```

Least Common Multiple (LCM)

```cpp
int LCM(int num1, int num2) {
    return ((num1 * num2) / GCD(num1, num2));
}
```

GCD and LCM in C++17

```cpp
#include <numeric>
int main () {
    int a = 48, b = 18;
    int gcd = std::gcd(a, b);
    int lcm = std::lcm(a, b);
    return 0;
}
```

Priority Queue (std::priority_queue)

Description: Demonstrates the use of C++ built-in `std::priority_queue` (from `<queue>`), including default behavior (Max-Heap), built-in Min-Heap, and Custom Struct Sorting. - **Default:** Max-Heap (largest element at the top). - **Built-in Min-Heap:** Uses `std::greater` (smallest element at the top). Very common with `std::pair` in graph algorithms. - **Custom Struct Sorting:** Overload the `<` operator inside a custom `struct`.

```cpp
#include <queue>
#include <vector>
#include <iostream>
#include <utility> // for pair

using namespace std;

// Struct with custom sorting rules
struct Item {
    int id;
    int priority;

    // Custom sorting rule: sort by priority
    ↪ ascending (min-heap)
    // If priority is equal, sort by id ascending
    ↪ (smaller id at the top)
    // Note: Since priority_queue is a Max-Heap by
    ↪ default, we invert the comparison logic.
    bool operator<(const Item& other) const {
        if (priority != other.priority) {
            return priority > other.priority; //
            ↪ Larger priority means "less" priority
            ↪ in max-heap (so smaller comes to top)
        }
        return id > other.id; // Larger ID means
        ↪ "less" priority (so smaller ID comes to
        ↪ top)
    }
};

void priorityQueueUsage() {
    priority_queue<int> max_pq;
```

```
    max_pq.push(10);
    max_pq.push(30);
    max_pq.push(20);
    // max_pq.top() is 30 (largest element first)

    priority_queue<int, vector<int>, greater<int>>
 ↪  min_pq;
    min_pq.push(10);
    min_pq.push(30);
    min_pq.push(20);
    // min_pq.top() is 10 (smallest element first)

    priority_queue<pair<int, int>, vector<pair<int,
 ↪  int>>, greater<pair<int, int>>> pair_pq;
    pair_pq.push({5, 101}); // {distance, node}
    pair_pq.push({2, 102});
    pair_pq.push({5, 100});
    // pair_pq.top() is {2, 102} (smallest distance
     ↪  first, then smallest node index)

    priority_queue<Item> custom_pq;
    custom_pq.push({101, 5});
    custom_pq.push({102, 5});
    custom_pq.push({103, 2});
    // custom_pq.top() is {103, 2} (priority 2), then
     ↪  {101, 5} (priority 5, smaller id first)
}
```

2. Prime Numbers

Count Primes (Sieve of Eratosthenes)

```
#include <vector>
#include <algorithm>

using namespace std;

int countPrimes(int N) {
    if (N < 2) return 0;
    vector<bool> isPrime(N + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= N; ++i) {
        if (isPrime[i]) {
            for (int j = i * i; j <= N; j += i) {
                isPrime[j] = false;
            }
        }
    }
    return count(isPrime.begin(), isPrime.end(),
     ↪  true);
}
```

Prime Number Test

```
bool isPrime(long long n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (long long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) {
            return false;
        }
    }
    return true;
}
```

List Primes (Sieve of Eratosthenes)

```
#include <vector>
using namespace std;
vector<int> sieve(int N) {
    vector<bool> isPrime(N + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= N; ++i) {
        if (isPrime[i]) {
            for (int j = i * i; j <= N; j += i) {
                isPrime[j] = false;
            }
        }
    }
    vector<int> primes;
    for (int i = 2; i <= N; ++i) {
        if (isPrime[i]) {
            primes.push_back(i);
        }
    }
    return primes;
}
```

Prime Factorization

```
#include <vector>
#include <utility>
using namespace std;
vector<pair<long long, int>> primeFactorization(long
 ↪  long n) {
    vector<pair<long long, int>> factors;
    for (long long i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            int count = 0;
            while (n % i == 0) {
                count++;
                n /= i; // Eliminate the factor
            }
            factors.push_back({i, count});
        }
    }
    if (n > 1) {
        factors.push_back({n, 1}); // Remaining prime
 ↪  factor
    }
    return factors;
}
```

3. String Processing

String Split (Regex)

Description: Splits a string into substrings based on a regular expression pattern delimiter. **Parameters:** - `str`: The target string to split. - `keys`: Regex pattern specifying the delimiters (default matches dashes `-` or commas `,`). - `keep`: If `true`, retains the matched delimiters as separate tokens in the output. **Usage:** Ideal for parsing complex string inputs. Requires `<regex>`.

```
#include <vector>
#include <string>
#include <regex>

using namespace std;

vector<string> split(const string& str, string keys =
 ↪  "(-{1,3}|,)", bool keep = false) {
    vector<string> tokens;
```

```
    regex re(keys);
    sregex_token_iterator it;
    if (keep) {
        it = sregex_token_iterator(str.begin(), str.end(), re, {-1, 0});
    } else {
        it = sregex_token_iterator(str.begin(), str.end(), re, -1);
    }
    sregex_token_iterator end;
    while (it != end) {
        tokens.push_back(*it);
        ++it;
    }
    return tokens;
}
```

Knuth-Morris-Pratt (KMP) Substring Search

Description: High-performance pattern searching algorithm. Precomputes the Longest Prefix Suffix (LPS) table to skip redundant character comparisons, eliminating the need to backtrack the text index. **Time Complexity:** $O(N + M)$ where N is text length, M is pattern length. **Space Complexity:** $O(M)$ for the LPS table. **Usage:** Fast substring matching when brute-force $O(N \cdot M)$ is too slow.

```
#include <vector>
#include <string>

using namespace std;

// Build the Longest Prefix which is also a Suffix (LPS) table
vector<int> buildLPS(const string &pattern) {
    int m = pattern.size();
    vector<int> lps(m, 0);
    int len = 0; // Length of the previous longest prefix suffix
    for (int i = 1; i < m; ) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1]; // Backtrack in pattern
            } else {
                lps[i++] = 0;
            }
        }
    }
    return lps;
}

// KMP search main function: Returns starting indices of matches
vector<int> KMPsearch(const string &text, const string &pattern) {
    vector<int> result;
    int n = text.size(), m = pattern.size();
    if (m == 0) return result;
    vector<int> lps = buildLPS(pattern);
    int i = 0, j = 0; // i -> text index, j -> pattern index
    while (i < n) {
        if (text[i] == pattern[j]) {
            i++; j++;
            if (j == m) {
                result.push_back(i - j); // Found match at index (i - j)
                j = lps[j - 1];
            }
        } else {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return result;
}
```

4. Computational Geometry

Closest Pair of Points (Divide & Conquer)

Description: Finds the closest pair of points in a 2D Euclidean plane using a divide-and-conquer strategy. 1. **Sort:** Sort all points by X-coordinates. 2. **Divide & Solve:** Recursively find the minimum distance in left and right halves. 3. **Combine:** Check points within a vertical strip of width $2d$ around the midline. Sort them by Y-coordinates and compare each point with at most 6 subsequent points. **Time Complexity:** $O(N \log N)$
Space Complexity: $O(N)$ **Usage:** Standard high-performance solution for closest-pair problems.

```
#include <vector>
#include <cmath>
#include <algorithm>
#include <limits>
#include <utility>

using namespace std;

struct Point {
    double x, y;
};

// Calculate Euclidean distance between two points
double dist(const Point& a, const Point& b) {
    return hypot(a.x - b.x, a.y - b.y);
}

// Brute force method for 3 or fewer points
double bruteForce(vector<Point>& points, int left, int right, pair<Point, Point>& bestPair) {
    double minDist = numeric_limits<double>::max();
    for (int i = left; i <= right; ++i) {
        for (int j = i + 1; j <= right; ++j) {
            double d = dist(points[i], points[j]);
            if (d < minDist) {
                minDist = d;
                bestPair = {points[i], points[j]};
            }
        }
    }
    return minDist;
}

// Check the strip area for closer points crossing the divide boundary
double stripClosest(vector<Point>& strip, double d, pair<Point, Point>& bestPair) {
    double minDist = d;
```

```
    sort(strip.begin(), strip.end(), [](const Point& a, const Point& b) {
        return a.y < b.y;
    });
    for (size_t i = 0; i < strip.size(); ++i) {
        // Compare each point with next points inside the strip
        for (size_t j = i + 1; j < strip.size() && (strip[j].y - strip[i].y) < minDist; ++j) {
            double newDist = dist(strip[i], strip[j]);
            if (newDist < minDist) {
                minDist = newDist;
                bestPair = {strip[i], strip[j]};
            }
        }
    }
    return minDist;
}

// Recursive Divide & Conquer helper utility
double closestUtil(vector<Point>& pointsSortedByX, int left, int right, pair<Point, Point>& bestPair) {
    // Use brute force if points count is 3 or less
    if (right - left <= 3) {
        return bruteForce(pointsSortedByX, left, right, bestPair);
    }
    int mid = (left + right) / 2;
    Point midPoint = pointsSortedByX[mid];
    pair<Point, Point> leftPair, rightPair;
    double dl = closestUtil(pointsSortedByX, left, mid, leftPair);
    double dr = closestUtil(pointsSortedByX, mid + 1, right, rightPair);
    double d = dl;
    bestPair = leftPair;
    if (dr < dl) {
        d = dr;
        bestPair = rightPair;
    }
    // Gather points inside vertical strip of width 2d
    vector<Point> strip;
    for (int i = left; i <= right; ++i) {
        if (abs(pointsSortedByX[i].x - midPoint.x) < d) {
            strip.push_back(pointsSortedByX[i]);
        }
    }
    pair<Point, Point> stripPair;
    double dStrip = stripClosest(strip, d, stripPair);
    if (dStrip < d) {
        d = dStrip;
        bestPair = stripPair;
    }
    return d;
}

// Main entry function to find the closest pair of points
pair<Point, Point> findClosestPair(vector<Point>& points) {
    vector<Point> pointsSortedByX = points;
    sort(pointsSortedByX.begin(), pointsSortedByX.end(), [](const Point& a, const Point& b) {
        return a.x < b.x;
    });
    pair<Point, Point> bestPair;
    closestUtil(pointsSortedByX, 0, pointsSortedByX.size() - 1, bestPair);
    return bestPair;
}
```

5. Permutation Operations

Next Permutation (Array-based)

```
#include <vector>
#include <algorithm>

using namespace std;

// Returns the lexicographically next permutation of the array.
vector<int> nextPermutation(vector<int> nums) {
    int n = nums.size();
    int i = n - 2;
    // Find the first decreasing element from the right
    while (i >= 0 && nums[i] >= nums[i + 1]) {
        i--;
    }
    if (i >= 0) {
        // Find the successor to nums[i] from the right
        int j = n - 1;
        while (nums[j] <= nums[i]) {
            j--;
        }
        swap(nums[i], nums[j]);
    }
    // Reverse the remaining ascending suffix
    reverse(nums.begin() + i + 1, nums.end());
    return nums;
}
```

Permutation Rank (Lexicographical Rank)

```
#include <vector>
#include <algorithm>

using namespace std;

// Binary Indexed Tree (Fenwick Tree) helper
struct FenwickTree {
    int n;
    vector<int> tree;
    FenwickTree(int n) : n(n), tree(n + 1, 0) {}
    void update(int i, int delta) {
        for (; i <= n; i += i & -i) tree[i] += delta;
    }
    int query(int i) {
        int sum = 0;
        for (; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }
};

// Calculates the 1-based lexicographical rank of a permutation.
// Assumes unique elements. Returns rank modulo MOD (default is 1e9 + 7).
long long getPermutationRank(vector<int> nums, long long MOD = 1000000007) {
    int n = nums.size();
    if (n == 0) return 1;

```

```
    // Precompute factorials modulo MOD
    vector<long long> fact(n, 1);
    for (int i = 1; i < n; ++i) {
        fact[i] = (fact[i - 1] * i) % MOD;
    }

    // Coordinate compression to map values to [1, n]
    vector<int> temp = nums;
    sort(temp.begin(), temp.end());
    vector<int> compressed(n);
    for (int i = 0; i < n; ++i) {
        compressed[i] = lower_bound(temp.begin(),
↪   temp.end(), nums[i]) - temp.begin() + 1;
    }

    FenwickTree bit(n);
    for (int i = 1; i <= n; ++i) {
        bit.update(i, 1);
    }

    long long rank = 1;
    for (int i = 0; i < n; ++i) {
        // Count how many unused elements to the
        ↪  right are smaller than compressed[i]
        int smallerCount = bit.query(compressed[i] -
        ↪  1);
        rank = (rank + smallerCount * fact[n - 1 -
↪   i]) % MOD;
        // Mark current element as used
        bit.update(compressed[i], -1);
    }
    return rank;
}
```

6. Math

Euler Totient Function

Description: Calculates Euler's totient function $\phi(x)$, which counts the number of positive integers strictly less than x that are coprime to x. **Time Complexity:** $O(\sqrt{x})$
Space Complexity: $O(1)$ **Usage:** Commonly used in number theory, particularly for finding modular multiplicative inverses and applying Euler's theorem.

```
#include <cmath>

int Phi(int x) {
    if (x < 2) return 0;
    int ret = x;
    int sq = sqrt(x);
    for (int p = 2; p <= sq; p++) {
        if (x % p == 0) {
            while (x % p == 0) x /= p;
            ret -= ret / p;
        }
        if (x == 1) break;
    }
    if (x > 1) ret -= ret / x;
    return ret;
}
```

7. Range Queries

Difference Array

Description: Efficiently applies multiple range addition updates $[L, R]$ by modifying only the boundaries of a difference array. A prefix sum is then used to reconstruct the final array values. **Returns:** The minimum coverage value across all positions after applying all updates. **Time Complexity:** $O(M+N)$ where M is the number of updates and N is the array size. **Space Complexity:** $O(N)$ for the difference array. **Usage:** Ideal for scenarios requiring many range updates followed by a single full-array query or state evaluation.

```
#include <iostream>
#include <vector>
#include <climits>

using namespace std;

long long differenceArray() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long N, M;
    if (!(cin >> N >> M)) return -1;

    // Size N+2 to prevent R+1 from exceeding
    ↪  boundary
    vector<long long> diff(N + 2, 0LL);

    // Read interval [L, R] for each turret
    for (int i = 0; i < M; i++) {
        long long L, R;
        cin >> L >> R;
        // Difference array update: +1 at start L, -1
        ↪  at position after end R+1
        diff[L] += 1;
        if (R + 1 <= N) {
            diff[R + 1] -= 1;
        }
    }

    // Compute prefix-sum directly on diff array and
    ↪  track the minimum value
    long long cover = 0;          // Current
    ↪  cumulative coverage at the current position
    long long answer = LLONG_MAX; // Stores the
    ↪  minimum coverage value

    for (int pos = 1; pos <= N; pos++) {
        cover += diff[pos]; // diff[pos] indicates
↪   the change in coverage at pos
        // cover represents the actual coverage count
        ↪  at pos
        if (cover < answer) {
            answer = cover;
        }
    }

    // Under normal circumstances, answer is at least
    ↪  0.
    // Return the answer.
    return answer;
}
```

Fenwick Tree (Binary Indexed Tree)

Ddescription: Support Point Update, Prefix Sum Query, Range Sum Query **Complexity:** Update : O(logN), Query : O(logN), Memory : O(N)

```
struct BIT {
    int n;
    vector<long long> bit;

```

```
    BIT(int _n) {
        n = _n;
        bit.assign(n + 1, 0);
    }

    void add(int idx, long long val) {
        while (idx <= n) {
            bit[idx] += val;
            idx += idx & -idx;
        }
    }

    long long sum(int idx) {
        long long res = 0;

        while (idx > 0) {
            res += bit[idx];
            idx -= idx & -idx;
        }

        return res;
    }

    long long query(int l, int r) {
        return sum(r) - sum(l - 1);
    }
};
```

Note: - lowbit[i] returns the size of the range managed by index i:

```
lowbit(x) = x & -x
```

- tree[i] stores the sum for the range managed by index

8. Graph Algorithms

Dijkstra's Algorithm (Single-Source Shortest Path)

```
#include <vector>
#include <queue>
#include <climits>

using namespace std;

const long long INF = LLONG_MAX;

// adj[u] = list of {v, weight} representing edge u -> v with given weight
vector<long long> dijkstra(int n, vector<vector<pair<int, long long>>>& adj, int src) {
    vector<long long> dist(n + 1, INF);
    dist[src] = 0;

    // {distance, node}, min-heap by distance
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if (d > dist[u]) continue; // Outdated entry, skip

        for (auto& [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }

    return dist; // dist[i] = shortest distance from src to i (INF if unreachable)
}
```

Build example:

```
int n = 5;
vector<vector<pair<int, long long>>> adj(n + 1); // 1-indexed, size n+1

auto addEdge = [&](int u, int v, long long w) {
    adj[u].push_back({v, w});
    adj[v].push_back({u, w}); // remove this line if the graph is directed
};

addEdge(1, 2, 4);
addEdge(1, 3, 1);
addEdge(3, 2, 2);

vector<long long> dist = dijkstra(n, adj, 1);
// dist[2] = 3 (via 1 -> 3 -> 2)
```

Bellman-Ford Algorithm (Handles Negative Weights + Cycle Detection)

```
#include <vector>
#include <climits>

using namespace std;

const long long INF = LLONG_MAX;

struct Edge {
    int u, v;
    long long w;
};

// Returns {dist, hasNegativeCycle}
// dist[i] = shortest distance from src to i (INF if unreachable)
pair<vector<long long>, bool> bellmanFord(int n, vector<Edge>& edges, int src) {
    vector<long long> dist(n + 1, INF);
    dist[src] = 0;

    // Relax all edges V-1 times
    for (int i = 0; i < n - 1; i++) {
        for (auto& e : edges) {
            if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
            }
        }
    }

    // Check for negative weight cycle: if any edge can still be relaxed
    bool hasNegativeCycle = false;
    for (auto& e : edges) {
        if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
            hasNegativeCycle = true;
            break;
        }
    }

```

```
    return {dist, hasNegativeCycle};
}
```

Build example:

```
int n = 5;
vector<Edge> edges;

// Just push every edge into a flat vector, no grouping needed
edges.push_back({1, 2, 4});
edges.push_back({1, 3, 1});
edges.push_back({3, 2, 2});
edges.push_back({2, 4, -3}); // negative weight is fine

auto [dist, hasNegCycle] = bellmanFord(n, edges, 1);

if (hasNegCycle) {
    // Negative cycle exists, shortest path is undefined
} else {
    // dist[4] = shortest distance
}
```

Floyd-Warshall Algorithm (All-Pairs Shortest Path)

```
#include <vector>
#include <climits>

using namespace std;

const long long INF = LLONG_MAX / 2; // Avoid overflow when adding two INF values

// dist[i][j] initial state should be set by caller:
// dist[i][i] = 0, dist[i][j] = edge weight if edge exists, else INF
// Returns true if graph has no negative cycle, false otherwise
bool floydWarshall(int n, vector<vector<long long>>& dist) {
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    // Check for negative cycle: if any dist[i][i] < 0
    for (int i = 1; i <= n; i++) {
        if (dist[i][i] < 0) return false; // Negative cycle detected
    }

    return true; // No negative cycle, dist is valid
}
```

Build example:

```
int n = 4;
vector<vector<long long>> dist(n + 1, vector<long long>(n + 1, INF));

// Initialize the diagonal
for (int i = 1; i <= n; i++) dist[i][i] = 0;

// Add edges (directed graph example)
dist[1][2] = 3;
dist[2][3] = 1;
dist[1][3] = 8; // if both a direct edge and a shorter indirect path exist, the algorithm resolves it automatically
dist[3][4] = 2;

bool ok = floydWarshall(n, dist);

if (!ok) {
    // Negative cycle exists
} else {
    // dist[1][4] = 6 (via 1 -> 2 -> 3 -> 4 = 3+1+2)
}
```

Minimum Spanning Tree (Kruskal's Algorithm)

Description: Finds the Minimum Spanning Tree (MST) of a connected, undirected, weighted graph. It uses the greedy approach: sorts all edges in non-decreasing order of their weight, and adds edges one by one if they don't form a cycle. A Disjoint Set Union (DSU) structure is used to check and manage cycles efficiently. **Time Complexity:** $O(E \log E + E\alpha(V))$ where E is the number of edges, V is the number of vertices, and α is the Inverse Ackermann function. **Space Complexity:** $O(V)$ for the DSU parent and rank arrays. **Usage:** Suitable for sparse graphs. Supports 1-indexed vertices by default.

```
#include <vector>
#include <algorithm>

using namespace std;

// Representation of an edge in the graph
struct Edge {
    int u, v;    // u, v: The two endpoint vertices (nodes) connected by this edge
    long long w; // w: The weight (cost/distance) of this edge
    // Overload the < operator to sort edges by weight
    bool operator<(const Edge& other) const {
        return w < other.w;
    }
};

// Disjoint Set Union (DSU) / Union-Find structure
struct DSU {
    vector<int> parent; // parent[i] stores the parent/leader of element i
    vector<int> rank;   // rank[i] stores the approximate depth of the tree rooted at i (for Union by Rank optimization)

    DSU(int n) { // n: The number of vertices/elements in the graph
        parent.resize(n + 1);
        rank.resize(n + 1, 0);
        for (int i = 0; i <= n; i++) {
            parent[i] = i; // Initially, every vertex is its own parent (disjoint set)
        }
    }

```

```
    // Finds the representative (root leader) of the set containing element i
    int find(int i) {
        if (parent[i] == i)
            return i;
        return parent[i] = find(parent[i]); // Path compression optimization
    }

    // Merges the set containing element i with the set containing element j.
    // Returns true if they were in different sets and successfully merged; false if they were already in the same set.
    bool unite(int i, int j) {
        int root_i = find(i);
        int root_j = find(j);
        if (root_i != root_j) {
            // Union by rank optimization: attach the smaller tree under the larger tree
            if (rank[root_i] < rank[root_j]) {
                parent[root_i] = root_j;
            } else if (rank[root_i] > rank[root_j]) {
                parent[root_j] = root_i;
            } else {
                parent[root_j] = root_i;
                rank[root_i]++;
            }
            return true; // Successfully merged
        }
        return false; // Already in the same set (connecting them would form a cycle)
    }
};

// Computes the MST weight and optionally stores the selected edges.
// n: Number of vertices (nodes) in the graph.
// edges: Input vector containing all edges in the graph (will be sorted in-place).
// mstEdges: Output vector to store the selected edges that form the MST.
// Returns the total MST weight, or -1 if the graph is disconnected (no MST exists).
long long kruskal(int n, vector<Edge>& edges, vector<Edge>& mstEdges) {
    mstEdges.clear();
    sort(edges.begin(), edges.end()); // Sort edges in ascending order of weight (greedy approach)

    DSU dsu(n); // Disjoint Set Union to detect cycles
    long long mst_weight = 0; // Accumulator for the total weight of the MST
    int edges_used = 0; // Counter for the number of edges added to the MST

    for (const auto& edge : edges) {
        // Try to connect the two endpoints of the current edge (edge.u and edge.v)
        if (dsu.unite(edge.u, edge.v)) {
            mst_weight += edge.w;
            mstEdges.push_back(edge);
            edges_used++;
            // A tree with n vertices always has exactly n - 1 edges
            if (edges_used == n - 1) {
                break;
            }
        }
    }

    if (edges_used == n - 1) {
        return mst_weight;
    }
    return -1; // Graph is not fully connected (could not select n - 1 edges)
}
```

Bipartite Matching (Hungarian Algorithm)

Finds the maximum matching in a bipartite graph. Nodes in set U are indexed 0 to $n-1$. Nodes in set V are indexed 0 to $m-1$.

```
#include <vector>

struct Hungarian {
    int n, m;
    std::vector<std::vector<int>> adj; // adj[u] contains neighbors in V
    std::vector<int> match_v;          // match_v[v] stores the matched u node
    std::vector<bool> visited;

    Hungarian(int n, int m) : n(n), m(m), adj(n), match_v(m, -1), visited(m, false) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    // DFS to find an augmenting path
    bool dfs(int u) {
        for (int v : adj[u]) {
            if (visited[v]) continue;
            visited[v] = true;

            // If v is not matched, or its match can find another choice
            if (match_v[v] == -1 || dfs(match_v[v])) {
                match_v[v] = u;
                return true;
            }
        }
        return false;
    }

    // Returns the maximum matching size
    int solve() {
        int ans = 0;
        std::fill(match_v.begin(), match_v.end(), -1);
        for (int i = 0; i < n; ++i) {
            std::fill(visited.begin(), visited.end(), false);
            if (dfs(i)) ans++;
        }
        return ans;
    }
};
```

9. Sequence Algorithms

Longest Increasing Subsequence (LIS)

Finds the length of the longest strictly increasing subsequence in $O(N \log N)$ time and $O(N)$ space.

```
#include <vector>
#include <algorithm>

```

```cpp
// Returns the length of the LIS
int getLIS(const std::vector<int>& nums) {
    if (nums.empty()) return 0;

    std::vector<int> v;
    for (int x : nums) {
        auto it = std::lower_bound(v.begin(), v.end(), x);
        if (it == v.end()) {
            v.push_back(x); // x is larger than all elements, extend the sequence
        } else {
            *it = x; // Replace the first element >= x to maintain a smaller tail
        }
    }
    return v.size();
}
```

10. Trie (Prefix Tree)

Description: A Trie (Prefix Tree) efficiently stores a collection of strings by sharing common prefixes. It supports insertion, exact string lookup, and prefix queries. **Time Complexity:** $O(L)$ for **insert**, **search**, and **startsWith**, L : The length of the string. **Space Complexity:** $O(\Sigma L)$, ΣL : The total number of inserted characters.

```cpp
#include <vector>
#include <string>

using namespace std;

struct Trie {
    // Each Trie node stores:
    // nxt[i] : the index of the child corresponding to character ('a' + i)
    // isEnd  : true if a word ends at this node
    struct Node {
        int nxt[26];
        bool isEnd;

        Node() {
            fill(nxt, nxt + 26, -1);
            isEnd = false;
        }
    };

    vector<Node> trie;

    Trie() {
        trie.emplace_back(); // Node 0 is the root
    }

    // Inserts a string into the Trie.
    void insert(const string& word) {
        int current = 0;

        for (char ch : word) {
            int index = ch - 'a';

            // Create a new node if the path does not exist
            if (trie[current].nxt[index] == -1) {
                trie[current].nxt[index] = trie.size();
                trie.emplace_back();
            }

            current = trie[current].nxt[index];
        }

        trie[current].isEnd = true;
    }

    // Returns true if the exact word exists in the Trie.
    bool search(const string& word) {
        int current = 0;

        for (char ch : word) {
            int index = ch - 'a';

            if (trie[current].nxt[index] == -1)
                return false;

            current = trie[current].nxt[index];
        }

        return trie[current].isEnd;
    }

    // Returns true if there exists any word with the given prefix.
    bool startsWith(const string& prefix) {
        int current = 0;

        for (char ch : prefix) {
            int index = ch - 'a';

            if (trie[current].nxt[index] == -1)
                return false;

            current = trie[current].nxt[index];
        }

        return true;
    }
};
```

11. Aho-Corasick Automaton (AC Automaton)

Description: An extension of Trie that supports efficient **multiple pattern matching** by adding failure links (similar to KMP). It can find all occurrences of multiple patterns in a text in linear time.

Use AC Automaton when:

- Matching **multiple patterns** in a single text
- Sensitive word filtering
- Dictionary matching
- DNA sequence matching
- String DP (Automaton DP)

If there is only **one pattern**, KMP is usually sufficient.

Algorithm Steps

```
Patterns -> Build Trie -> Build Failure Links (BFS) -> Scan Text
```

Failure links allow the automaton to continue matching from the **longest valid suffix** instead of restarting from the root.

```cpp
struct AhoCorasick {
    static constexpr int SIGMA = 26;

    struct Node {
```

```
        int nxt[SIGMA];
        int fail;
        vector<int> out;

        Node() {
            memset(nxt, 0, sizeof(nxt));
            fail = 0;
        }
    };

    vector<Node> trie;

    AhoCorasick() {
        trie.emplace_back();   // root
    }

    void insert(const string &s, int id) {
        int cur = 0;
        for (char c : s) {
            int x = c - 'a';
            if (!trie[cur].nxt[x]) {
                trie[cur].nxt[x] = trie.size();
                trie.emplace_back();
            }
            cur = trie[cur].nxt[x];
        }
        trie[cur].out.push_back(id);
    }

    void build() {
        queue<int> q;

        for (int c = 0; c < SIGMA; c++) {
            int v = trie[0].nxt[c];
            if (v) q.push(v);
        }

        while (!q.empty()) {
            int u = q.front();
            q.pop();

            for (int c = 0; c < SIGMA; c++) {
                int &v = trie[u].nxt[c];

                if (v) {
                    trie[v].fail =
↪   trie[trie[u].fail].nxt[c];
                    trie[v].out.insert(
                        trie[v].out.end(),

↪   trie[trie[v].fail].out.begin(),
                        trie[trie[v].fail].out.end()
                    );
                    q.push(v);
                } else {
                    v = trie[trie[u].fail].nxt[c];
                }
            }
        }
    }

    vector<int> query(const string &text) {
        vector<int> res;
        int cur = 0;

        for (char c : text) {
            cur = trie[cur].nxt[c - 'a'];
            for (int id : trie[cur].out)
                res.push_back(id);
        }

        return res;
    }
};
```

Procedure

```
1. Insert all patterns into the Trie.
2. Build failure links using BFS.
3. Scan the text once.
4. Report every matched pattern.
```

Notes

- Root index is usually `0`.
- Failure links are built using **BFS**.
- Missing transitions are completed during `build()`, so matching is **O(|Text|)**.
- `out` stores all pattern IDs ending at the current node.
- Frequently combined with **DP** to solve forbidden-string and string-counting problems.

12. Expected Value DP

Idea: Define the expectation of each state and write the expectation equation directly.

General Formula

$$E(s) = \sum P_i \left(C_i + E(next_i)\right)$$

where

- P_i: probability of transition
- C_i: immediate cost
- $E(next_i)$: future expectation

If Self-loop Exists

Suppose

$$E(s) = A + pE(s)$$

Move the self-loop term to the left:

$$E(s) = \frac{A}{1-p}$$

This is the most common transformation in Expected Value DP.

Steps

1. Define the DP state.
2. Enumerate all possible transitions.
3. Write the expectation equation directly.
4. Move self-loop terms to the left if necessary.
5. Solve the equation.
6. Use memoization or DP.

Common State Types

- `dp[i]`
- `dp[mask]`
- `dp[x][y]`
- `dp[a][b][c]`